Coercion-Resistant Protocols for Governance Under Authoritarian Surveillance

Pars Network Technical Whitepaper PNP-005

Cyrus Pahlavi, Pars Team research@pars.network Version 1.0 — February 2026

Abstract

We present a suite of coercion-resistant protocols for the Pars Network, designed to protect diaspora community members who participate in decentralised governance under threat of physical coercion by authoritarian regimes. Standard cryptographic protocols assume users freely control their keys; this assumption fails under detention, threat, or violence. We introduce four mechanisms: (1) deniable authentication producing transcripts indistinguishable from innocent interactions; (2) panic passwords triggering credential rotation and evidence destruction while presenting a plausible decoy environment; (3) decoy wallets with credible on-chain histories; and (4) plausible deniability for governance votes through re-randomisable ballot encryption. We formalise coercion resistance in the Universal Composability framework, prove receipt-freeness and coercion resistance under standard assumptions, and present a threat-scenario analysis based on documented practices of surveillance states. The protocols integrate with the Pars session layer (PNP-003) and governance system (PNP-001).

Keywords: coercion resistance, deniable authentication, panic passwords, decoy wallets, plausible deniability, governance privacy

Contents

1 Introduction **3**
1.1 Coercion Scenarios . . . 3
1.2 Design Goals . . . 3
1.3 Contributions . . . 3

2 Threat Model **3**

3 Deniable Authentication **4**
3.1 Problem . . . 4
3.2 Ring-Signature-Based Authentication . . . 4
3.3 PSP Integration . . . 4

4 Panic Passwords **4**
4.1 Concept . . . 4
4.2 Dual-Key Derivation . . . 5
4.3 Panic Actions . . . 5

5 Decoy Wallets **6**
5.1 Requirements . . . 6
5.2 Construction . . . 6
5.3 Funding Unlinkability . . . 7

6 Receipt-Free Governance Voting **7**
6.1 Re-Randomisable Ballots . . . 7
6.2 Voting Protocol . . . 7
6.3 Trustee Threshold . . . 8

7 Plausible Deniability for Governance **8**
7.1 Vote Deniability . . . 8

8 Dead Man's Switch **8**

9 Threat Scenario Analysis **9**

10 Implementation **9**
10.1 Secure Erasure 9
10.2 App Disguise 10

11 Formal Security Analysis **10**

12 Limitations **10**

13 Related Work **10**

14 Conclusion **10**

A Decoy Profiles **12**

B Panic Sequence **12**

1 Introduction

Participation in decentralised governance is a political act. For the Persian diaspora, especially those with family inside Iran, governance participation carries material risk. The Islamic Republic has a documented history of coercing individuals to reveal digital credentials, confiscating devices, and retaliating against diaspora activists' families [1].

Standard cryptographic threat models assume private keys are under sole owner control. This collapses under physical coercion.

1.1 Coercion Scenarios

S1: Border Crossing: Device inspected; user compelled to unlock and demonstrate contents.

S2: Detention: Interrogation demanding passwords, wallet keys, communication histories.

S3: Family Pressure: Threats against family members to compel vote record disclosure.

S4: Warrant: Cooperative jurisdiction compels provider to reveal participation records.

1.2 Design Goals

1. **Coercion Resistance:** Coerced user complies without revealing true governance participation or holdings.
2. **Receipt-Freeness:** No user can prove how they voted.
3. **Plausible Deniability:** Existence of true credentials indistinguishable from non-existence.
4. **Minimal Overhead:** $< 2\times$ overhead over normal operations.

1.3 Contributions

- Formalisation of coercion resistance for blockchain governance.
- Deniable authentication with offline deniability.
- Panic password system with cryptographic decoy generation.
- Decoy wallets with credible transaction histories.
- Re-randomisable ballot encryption for receipt-free voting.
- Threat-scenario analysis from documented state practices.

2 Threat Model

Definition 2.1 (Coercive Adversary)**.** $\mathcal{A}_{\text{coerce}}$ *has network adversary capabilities and can additionally:*

(a) Compel secret disclosure

(b) Compel any protocol action

(c) Inspect device state

(d) Observe physical behaviour during execution

Definition 2.2 (Coercion Resistance)**.** *Protocol* Π *is coercion-resistant if strategy* σ_{fake} *exists such that a coerced user following it produces interaction indistinguishable from any* σ'*, from* $\mathcal{A}_{\text{coerce}}$*'s perspective.*

Definition 2.3 (Receipt-Freeness)**.** *A voting protocol is receipt-free if no voter can construct a receipt proving their vote, even cooperating with the receipt requester.*

3 Deniable Authentication

3.1 Problem

Digital signatures produce non-repudiable transcripts. If coerced to reveal authentication history, signatures prove specific interactions.

3.2 Ring-Signature-Based Authentication

Algorithm 1 Deniable Authentication

Require: Prover A with (a, A), Verifier B with (b, B)

1: A generates ephemeral $(r, R = rG)$
2: $\sigma \leftarrow \text{RingSign}(\{A, B\}, a, m)$
3: $A \rightarrow B$: (m, R, σ)
4: B verifies: $\text{RingVerify}(\{A, B\}, m, \sigma)$
5: Third party cannot tell whether A or B signed

Theorem 3.1 (Deniability)**.** *Ring-signature authentication produces simulatable transcripts. No third party can determine which ring member signed.*

Proof. By ring signature anonymity [2], σ from A is computationally indistinguishable from σ from B. Verifier B can produce valid ring signatures using their own key, simulating any transcript. □

3.3 PSP Integration

PSP (PNP-003) already provides pairwise deniability through X3DH's use of DH. Ring-signature authentication extends this to explicit authentication contexts (governance proposals, identity assertions).

4 Panic Passwords

4.1 Concept

Definition 4.1 (Panic Password System)**.**

- *Primary password* p_1*: unlocks true environment*
- *Panic password* p_2*: unlocks decoy, triggers protection*
- $\text{KDF}(p_1) \neq \text{KDF}(p_2)$*; both produce valid-looking keys*

4.2 Dual-Key Derivation

Algorithm 2 Dual-Key Derivation

procedure SETUP(p_1, p_2)
  $k_1 \leftarrow \text{Argon2id}(p_1, s_1)$
  $k_2 \leftarrow \text{Argon2id}(p_2, s_2)$
  $E_1 \leftarrow \text{AES}(k_1, \text{state}_{\text{true}})$
  $E_2 \leftarrow \text{AES}(k_2, \text{state}_{\text{decoy}})$
  Store (E_1, E_2, s_1, s_2)
end procedure
procedure UNLOCK(p)
  Try $k \leftarrow \text{Argon2id}(p, s_1)$; if AES.Dec($k, E_1$) succeeds: return true env
  Try $k \leftarrow \text{Argon2id}(p, s_2)$; if AES.Dec($k, E_2$) succeeds: TriggerPanic(); return decoy env
  Return failure
end procedure

4.3 Panic Actions

1. **Key Rotation:** Rotate session keys and prekey bundles.
2. **Evidence Destruction:** Secure-erase primary state E_1.
3. **Silent Alert:** Dead-man notification via mesh to designated contacts.
4. **Decoy Presentation:** Show plausible decoy environment.

Listing 1: Panic Action Handler

```go
type PanicHandler struct {
    keys     *KeyManager
    storage  *SecureStorage
    mesh     *MeshClient
    contacts []PeerID
}

func (h *PanicHandler) Execute(
    ctx context.Context,
) error {
    // Rotate keys (best-effort)
    h.keys.RotateAll(ctx)

    // Destroy primary state
    if err := h.storage.SecureErase(
        "primary_state",
    ); err != nil {
        return fmt.Errorf("erase: %w", err)
    }

    // Silent alert via mesh
    alert := &CoercionAlert{
        Time: time.Now().Unix(),
        Type: AlertPanic,
    }
    for _, c := range h.contacts {
```

```
        h.mesh.SendAsync(ctx, c, alert)
    }
    return nil
}
```

Proposition 4.1 (Post-Panic Indistinguishability)**.** *After panic activation, an inspector cannot determine whether a primary state ever existed.*

Proof sketch. After secure erasure of E_1, only (E_2, s_1, s_2) remains. s_1 and zeroed E_1 space are indistinguishable from unused storage. The device appears to contain a single encrypted state (E_2), unlockable with the panic password presented as "the only" password. □

5 Decoy Wallets

5.1 Requirements

1. **Credibility:** Plausible balance and history.
2. **Verifiability:** Real on-chain transactions.
3. **Consistency:** Matches claimed usage pattern.
4. **Independence:** No on-chain link to true wallet.

5.2 Construction

Algorithm 3 Decoy Wallet Construction

procedure CreateDecoy(profile P)
  $(d, D) \leftarrow$ fresh key pair
  Fund via stealth address from decoy fund
  for $i = 1$ to P.txCount **do**
    Schedule tx at $t_{\text{now}} - P.\text{spacing} \cdot i$
    Amount $\sim P$.amountDist
    Recipient $\in P$.counterparties
  end for
  Maintain P.targetBalance
  Store under panic key k_2
end procedure

Listing 2: Decoy Profile

```
type DecoyProfile struct {
    MinBalance    *big.Int
    MaxBalance    *big.Int
    TxPerWeek     int
    AmountMean    *big.Int
    AmountStdDev  *big.Int
    Counterparties []common.Address
    Tokens        []TokenHolding
    VoteFreq      float64
}

var Standard = DecoyProfile{
    MinBalance: eth(0.1),
```

```
    MaxBalance: eth(2.0),
    TxPerWeek:  3,
    AmountMean: eth(0.05),
    AmountStdDev: eth(0.03),
    VoteFreq:   0.3,
}
```

5.3 Funding Unlinkability

1. Decoy funds from community-managed pool with ring-signature deposits [10].
2. Transfers via stealth addresses (PNP-007).
3. Pool replenished via constant-rate treasury drip.

6 Receipt-Free Governance Voting

6.1 Re-Randomisable Ballots

Definition 6.1 (Re-Randomisable Ballot)**.** *Ballot $B = (g^r,\ v \cdot h^r)$ under ElGamal with PK $h = g^x$ is re-randomised:*

$$B' = (g^{r+r'},\ v \cdot h^{r+r'})$$

by any party knowing h, using fresh r'.

6.2 Voting Protocol

Algorithm 4 Coercion-Resistant Vote Casting

Require: Voter V, vote v, election PK h

Registration (anonymous):
V obtains credential cred via ZK eligibility proof
V also registers fake credential $\text{cred}_{\text{fake}}$

Casting:
$B \leftarrow \text{ElGamal.Enc}(h, v, r)$
$\pi \leftarrow \text{Prove}(B \in \{\text{Enc}(h, 0), \text{Enc}(h, 1)\})$
Submit (B, π, cred) via anonymous channel

Re-randomisation (trustees):
for all trustee T_j **do**
    $B \leftarrow \text{ReRand}(B, r'_j)$
end for
After this, V cannot link to final ballot

Under coercion: cast with $\text{cred}_{\text{fake}}$
Tally system silently ignores fake-credential votes

Theorem 6.1 (Receipt-Freeness)**.** *After trustee re-randomisation, no voter can construct a receipt linking themselves to a re-randomised ballot.*

Proof. Original ballot $B = (g^r, v \cdot h^r)$ becomes $B' = (g^{r+r'}, v \cdot h^{r+r'})$ with $r' = \sum_j r'_j$ unknown to the voter. Even revealing original r, the voter cannot link B to B'. By ElGamal IND-CPA security, B' is indistinguishable from any ballot for the same vote value. □

Theorem 6.2 (Coercion Resistance)**.** *A coerced voter using the fake-credential strategy is indistinguishable from honest voting.*

Proof. $\text{cred}_{\text{fake}}$ is syntactically identical to cred (both are ZK proofs). The coercer cannot distinguish them without credential issuer access. The vote with $\text{cred}_{\text{fake}}$ is encrypted identically. The tally system silently rejects it; the coercer cannot detect rejection. □

6.3 Trustee Threshold

t-of-n threshold with recommended $t = 5$, $n = 9$, trustees from distinct jurisdictions.

Definition 6.2 (Threshold Re-Randomisation)**.** *Election PK $h = g^x$ with $x = \sum_{j=1}^{n} x_j \bmod q$ shared among n trustees. Re-randomisation requires $\geq t$ contributors.*

7 Plausible Deniability for Governance

7.1 Vote Deniability

1. **Credential Hiding:** Encrypted under primary key; destroyed on panic.
2. **No On-Chain Identity:** Votes via session layer, aggregated by trustees before L2 commitment.
3. **Time-Bounded Records:** Encryption keys rotated per epoch; individual data deleted after tally.

Proposition 7.1 (Participation Deniability)**.** *No adversary can determine with non-negligible advantage whether a specific user participated.*

Proof sketch. Ballots submitted via anonymous routing, re-randomised by trustees. No on-chain voter-ballot mapping. Device after panic contains no credential evidence. □

8 Dead Man's Switch

Algorithm 5 Dead Man's Switch

```
procedure SETUP(user U, interval T, contacts C)
    Register heartbeat schedule with interval T
    Pre-encrypt alert messages for each c ∈ C
    Distribute time-locked capsules to relay network
end procedure
procedure HEARTBEAT
    Authenticated heartbeat to relay; reset timer
end procedure
procedure TRIGGER
    Timer expires without heartbeat
    Decrypt and deliver alerts; rotate U's keys; publish DID revocation
end procedure
```

Definition 8.1 (Time-Lock Capsule)**.** $C = (N, a, t, Z)$ *where* $N = pq$, $Z = m \oplus H(a^{2^t} \bmod N)$. *Openable in $O(t)$ sequential squarings; not parallelisable [11].*

Table 1: Mitigation Matrix

Scenario	Threat	Mitigation
S1: Border	Device inspection	Decoy wallet + panic pw
S1: Border	Password demand	Panic pw as primary
S2: Detention	Extended interrogation	Panic destroys evidence
S2: Detention	Key disclosure	Decoy keys presented
S2: Detention	Missed check-in	Dead man's switch
S3: Family	Vote disclosure	Receipt-free voting
S3: Family	Coerced voting	Fake credentials
S4: Warrant	Server data request	No plaintext server-side
S4: Warrant	Blockchain analysis	Stealth addresses

9 Threat Scenario Analysis

10 Implementation

10.1 Secure Erasure

Listing 3: Secure Erasure

```go
func (s *SecureStorage) Erase(
    key string,
) error {
    path := filepath.Join(s.path, key)
    info, err := os.Stat(path)
    if err != nil {
        return fmt.Errorf("stat: %w", err)
    }
    size := info.Size()
    f, err := os.OpenFile(
        path, os.O_WRONLY, 0,
    )
    if err != nil {
        return fmt.Errorf("open: %w", err)
    }
    defer f.Close()

    buf := make([]byte, 4096)
    for pass := 0; pass < 3; pass++ {
        f.Seek(0, 0)
        rem := size
        for rem > 0 {
            n := min(int64(len(buf)), rem)
            rand.Read(buf[:n])
            f.Write(buf[:n])
            rem -= n
        }
        f.Sync()
    }
    f.Close()
    return os.Remove(path)
}
```

10.2 App Disguise

1. Icon and name changeable to common utility (calculator, weather).
2. Disguise activated via specific gesture.
3. Functional utility interface; Pars UI behind secondary gesture.

11 Formal Security Analysis

Definition 11.1 ($\mathcal{F}_{\mathrm{CR}}$)**.** *Ideal functionality accepting votes from registered voters. On coercion: accepts "coerced vote" without counting it, reveals same view as honest vote to coercer, tallies only non-coerced votes.*

Theorem 11.1 (UC Realisation)**.** *The Pars coercion-resistant voting protocol UC-realises $\mathcal{F}_{\mathrm{CR}}$ under DDH, assuming $< t$ corrupted trustees.*

The proof follows Juels et al. [4], extended with anonymous credential and session-layer integration.

12 Limitations

1. No protocol defeats sustained physical torture; operational security (ephemeral devices) is necessary for high-risk situations.
2. Decoy credibility requires regular profile updates.
3. If adversary knows panic system exists, they may demand both passwords.
4. $\geq t$ colluding trustees can de-anonymise votes.

13 Related Work

Juels et al. [4] introduced coercion-resistant e-voting. Civitas [5] implemented the scheme. Chaum [6] provides receipt-free verification. TrueCrypt/VeraCrypt [7, 8] pioneered plausible-deniability disk encryption. Briar [9] has a panic button. Pars integrates coercion resistance into a blockchain governance platform with diaspora-specific threat modelling.

14 Conclusion

The Pars coercion-resistant suite addresses physical security threats facing diaspora members. Deniable authentication, panic passwords, decoy wallets, and receipt-free voting provide layered defence. Formal analysis in the UC framework proves coercion resistance and receipt-freeness under standard assumptions.

Cryptographic security alone is insufficient when adversaries operate outside standard threat model assumptions. Explicitly modelling coercion as a protocol-level threat enables meaningful protection even when physical safety is compromised.

References

[1] Freedom House, "Freedom on the Net: Iran," 2023.

[2] R. Rivest, A. Shamir, and Y. Tauman, "How to Leak a Secret," *ASIACRYPT 2001*, pp. 552–565, 2001.

[3] N. Unger and I. Goldberg, "Deniable Key Exchanges for Secure Messaging," *CCS*, pp. 1211–1223, 2015.

[4] A. Juels, D. Catalano, and M. Jakobsson, "Coercion-Resistant Electronic Elections," *WPES*, pp. 61–70, 2005.

[5] M. Clarkson, S. Chong, and A. Myers, "Civitas: Toward a Secure Voting System," *IEEE S&P*, pp. 354–368, 2008.

[6] D. Chaum, "Secret-ballot receipts," *IEEE Security & Privacy*, 2(1), 2004.

[7] TrueCrypt Foundation, "TrueCrypt: Hidden Volumes," 2004.

[8] IDRIX, "VeraCrypt: Plausible Deniability," 2015.

[9] Briar Project, "Briar: Secure messaging, anywhere," 2017.

[10] S. Noether, "Ring Signature Confidential Transactions for Monero," *MRL-0005*, 2015.

[11] R. Rivest, A. Shamir, and D. Wagner, "Time-lock puzzles and timed-release crypto," *MIT/LCS/TR-684*, 1996.

[12] R. Canetti, "Universally Composable Security," *FOCS*, pp. 136–145, 2001.

[13] S. Goldwasser, S. Micali, and C. Rackoff, "The Knowledge Complexity of Interactive Proof Systems," *SIAM J. Comput.*, 18(1), 1989.

[14] D. Chaum, "Untraceable electronic mail," *Commun. ACM*, 24(2), 1981.

[15] T. Pedersen, "A Threshold Cryptosystem without a Trusted Party," *EUROCRYPT '91*, pp. 522–526, 1991.

[16] T. ElGamal, "A Public Key Cryptosystem Based on Discrete Logarithms," *IEEE Trans. Inf. Theory*, 31(4), 1985.

[17] D. Boneh, X. Boyen, and H. Shacham, "Short Group Signatures," *CRYPTO 2004*, pp. 41–55, 2004.

[18] J. Groth, "On the Size of Pairing-Based Non-interactive Arguments," *EUROCRYPT 2016*, pp. 305–326, 2016.

[19] W. Diffie and M. Hellman, "New directions in cryptography," *IEEE Trans. Inf. Theory*, 22(6), 1976.

[20] M. Marlinspike and T. Perrin, "The Signal Protocol," 2016.

[21] G. Danezis and I. Goldberg, "Sphinx," *IEEE S&P*, 2009.

[22] D. J. Bernstein, "Curve25519," *PKC 2006*, 2006.

[23] B. Adida, "Helios: Web-based Open-Audit Voting," *USENIX Security*, pp. 335–348, 2008.

[24] J. Benaloh, "Simple Verifiable Elections," *EVT*, 2006.

[25] Amnesty International, "Iran: Digital Surveillance," 2022.

[26] Reporters Without Borders, "Iran: Digital Security for Journalists," 2023.

A Decoy Profiles

Profile	Balance	Tx/Wk	Tokens
Minimal	0.01–0.1 PARS	1	0
Standard	0.1–2.0 PARS	3	2
Active	1.0–10.0 PARS	7	5

B Panic Sequence

1. Enter panic password
2. Derive decoy key
3. Background: rotate keys (100 ms)
4. Background: erase primary state (200 ms)
5. Background: mesh alerts (async)
6. Foreground: present decoy (<50 ms)
7. Total visible delay: <300 ms